

Native Windows reproduction package:

$R(C4,C6,C8)=12$

This package supplies three finite computational modules, represented by nine CNF/RUP pairs, and complete C++14 source for input reconstruction, proof generation and independent RUP checking. The graph-theoretic reductions and the remainder of the Ramsey argument are proved in the submitted paper. These programs do not formalize the entire mathematical proof.

Run

Extract the package to a writable directory on Windows x64. No Python, PySAT, separate SAT installation, Visual Studio or separately installed Visual C++ Redistributable is needed to run the supplied executables. The supplied Release programs use the static C/C++ runtime; their inspected DLL dependency is `KERNEL32.dll`. Windows PowerShell and the command processor are built-in tools used by the launchers.

Double-click **VERIFY_ALL.cmd** to reconstruct and compare all nine inputs, then check all nine archived proofs. Double-click **REPRODUCE_ALL.cmd** to generate new inputs, compare them with the archive, produce new proofs and check those proofs against the new inputs. Each window waits for a key when finished. Logs and new files go into a fresh timestamped `results/` directory.

For unattended use, run from a command prompt:

```
VERIFY_ALL.cmd --no-pause
REPRODUCE_ALL.cmd --no-pause
CHECK_HASHES.cmd --no-pause
```

A quick test, including the complete input audit, is:

```
VERIFY_ALL.cmd --no-pause --case B-leafK4
REPRODUCE_ALL.cmd --no-pause --case B-leafK4
```

For a complete run, the final summary must report all nine input audits and **9 selected certificate checks passed**, with exit code 0. A selected-case trial does not establish a complete 9/9 result. See `REPRODUCE_EN.txt` or `REPRODUCE_ZH.txt` for the full operating instructions.

Earlier validation reports in `reports/` retain historical measurements and executable identities. They do not automatically validate a later build. `reports/release_validation.json` records the checks performed for this release and the identity of its newly compiled solver. Fresh proofs need not have the same hashes, lengths or running times as the archived proofs.

Rebuild from source

Install the Visual Studio Desktop development with C++ workload and Windows SDK. Then double-click **BUILD_ALL.cmd**, or run:

```
BUILD_ALL.cmd --no-pause
VERIFY_ALL.cmd --no-pause --case B-leafK4 --bin rebuilt_bin
REPRODUCE_ALL.cmd --no-pause --case B-leafK4 --bin rebuilt_bin
```

BUILD_ALL.cmd copies the supplied sources to a new `results/build_TIMESTAMP_RANDOM/src/` directory and invokes each component's build command. Complete compiler output is retained as `inputs_build.log`, `checker_build.log` and `solver_build.log`, with `summary.txt` recording the overall result. All three programs must build successfully before they are copied to `rebuilt_bin/`. This command never replaces `bin/` or edits the distributed sources. Rebuilding may change executable hashes; test rebuilt programs with `--bin rebuilt_bin` rather than overwriting the distributed ones. The build uses C++14, optimized Release settings and `/MT` static runtime.

The command returns 0 only on complete success. Build failure, input mismatch, solver error or rejected proof produces a nonzero result. Solver exit 20 means UNSAT, but the reproduction command still requires the independent checker to return 0 against the generated input. Solver output alone is never accepted.

Contents and scope

Location	Contents
<code>bin/</code>	Three native Windows x64 Release executables
<code>data/</code>	Exactly nine original CNFs and their nine original RUP proofs
<code>src/inputs/</code>	Complete graph/auxiliary-clause reconstruction and VS project
<code>src/checker/</code>	Independent watched-literal RUP checker
<code>src/solver/</code>	Windows wrapper, adapted Glucose core, notices and source patch
<code>vendor/</code>	Original official Glucose 4.2.1 source archive
<code>docs/</code>	Input semantics, command reference, provenance and licensing notes
<code>reports/</code>	Historical validation evidence and the current release record
<code>instances.json</code>	Exact nine distributed CNF/RUP identities and SHA256 hashes
<code>SHA256SUMS.txt</code>	Hashes of the immutable distributed files

The finite modules comprise the two local arrow instances `48T1,48T2`, the three normalized ten-vertex hexagon cases, and the four normalized blue boundary cases. The meanings and auxiliary encodings are documented in `docs/INPUT_SEMANTICS.md`. The case-coverage arguments remain mathematical proofs in the paper. Published cycle theorems and McKay's external extremal values are separate dependencies and are not certified by these nine inputs.

`RamseyInputs` reconstructs every clause family and compares complete clause multisets, including all auxiliaries. `RamseyRup` checks each proof addition by unit propagation and requires an explicit final empty-clause addition. Neither program performs SAT search. `RamseySolve` generates proofs using the official Glucose core, whose algorithm and authorship are identified in `docs/PROVENANCE_AND_LICENSES.md`. The checker is not formally verified.

The programs and explanations were prepared with OpenAI Codex assistance. Different implementations are not claimed to have independent human authorship. The paper contains the AI-use statement and author responsibility declaration.

See `docs/WINDOWS_COMMANDS.md` for every option and the exact log layout. Generated results, build intermediates and rebuilt binaries are deliberately outside the immutable checksum manifest and are not included in this package.